

Lesson 2: C Programming Advanced Multi-Tasking, RTOS & OS Architecture

Thinh Tang - Trieu Nguyen

Bộ môn Điện tử - Viễn thông, Khoa Điện - Điện tử
Trường Đại học Bách khoa - ĐHQG-HCM

Email liên hệ: mliot@hcmut.edu.vn

Machine Learning & IoT Lab (ML IoT)
Ho Chi Minh City University of Technology (HCMUT)

1. Tổng quan MCU vs MPU
2. Cấu trúc dữ liệu: Struct, Union & Bit-fields
3. Phân vùng bộ nhớ MCU & Quản lý cấp phát động
4. Hệ thống Ngắt (Interrupt) & Cơ chế ISR
5. Đa luồng (Multithreading) & Bộ lập lịch (Scheduler)
6. Kỹ thuật lập trình: Con trỏ Hàm & Callback
7. Đồng bộ & Truyền thông: Mutex, Semaphore, Queue

Contact Information

Address: 403.1 BK.B6, HCMUT Campus 2

Email:

nkloi@hcmut.edu.vn

mlandiotlab@gmail.com

Website: mliotlab.com

Facebook: facebook.com/hcmut.ml.iot.lab

YouTube: youtube.com/@mliotlab

MCU (Vi điều khiển - All-in-One)

- ▶ Tích hợp **Bộ xử lý (Core)**, bộ nhớ **RAM**, bộ nhớ chương trình **Flash (ROM)**, và các ngoại vi **I/O** (GPIO, UART, ADC, SPI...) trên một chip đơn.
- ▶ Thường sử dụng trong hệ thống nhỏ, công suất thấp, giá rẻ, chạy hệ điều hành thời gian thực (RTOS) hoặc Bare-metal.
- ▶ Ví dụ: STM32 (ARM Cortex-M), ESP32, AVR.

MPU (Bộ vi xử lý - Core Only)

- ▶ Chỉ chứa **CPU / Processor Core** cùng bộ nhớ đệm Cache.
- ▶ **Không** tích hợp sẵn RAM, Flash hay ngoại vi I/O trên chip. Phải kết nối RAM ngoại (DDR) và bộ nhớ ngoài (eMMC, NAND Flash) qua các đường BUS tốc độ cao.
- ▶ Hiệu năng cao, chạy hệ điều hành đầy đủ như Linux, Windows.
- ▶ Ví dụ: Intel Core i7, ARM Cortex-A (Raspberry Pi).

Tiêu chí	MCU (Microcontroller)	MPU (Microprocessor)
Cấu trúc	Tích hợp sẵn Core, RAM, ROM, IO trên một chip	Chỉ có Core CPU; RAM, ROM, IO kết nối bên ngoài
Hiệu năng	Thấp - Trung bình (Vài MHz đến vài trăm MHz)	Cao (Hàng GHz, xử lý đa luồng)
Bộ nhớ (RAM/ROM)	Giới hạn (Vài KB đến vài MB)	Mở rộng linh hoạt (Hàng GB RAM, GB ROM ngoài)
Hệ điều hành	Chạy trực tiếp (Bare-metal) hoặc RTOS	Bắt buộc/Thường dùng hệ điều hành lớn (Linux, OS)
Ứng dụng	Điều khiển, giám sát cảm biến, IoT nhỏ	Máy tính, điện thoại, xử lý ảnh, AI Edge

Table: So sánh đặc tính MCU và MPU

Data Alignment (Căn chỉnh dữ liệu)

- ▶ CPU 32-bit đọc dữ liệu tốt nhất khi địa chỉ của biến chia hết cho kích thước của nó (ví dụ `uint32_t` nằm ở địa chỉ chia hết cho 4).
- ▶ Để làm được điều này, compiler chèn thêm các byte trống (**Padding bytes**) vào giữa các member.

Minh họa mã nguồn

```
// Sizeof = 8 bytes due to alignment constraints.  
// The 32-bit CPU accesses 2-byte or 4-byte boundaries faster.  
// Hence, the compiler inserts padding to align members.  
typedef struct {  
    uint8_t a;           // Offset 0 (1 byte)  
    // 1 byte padding automatically inserted here at offset 1  
    uint16_t b;          // Offset 2 (2 bytes)  
    uint32_t c;          // Offset 4 (4 bytes)  
} NormalStruct_t;  
  
// Sizeof = 7 bytes.  
// __attribute__((packed)) forces the compiler to omit all  
padding.  
// Good for network packets / hardware protocol payloads (  
exact match).  
// CPU might require multiple cycles to access unaligned  
members.  
typedef struct __attribute__((packed)) {  
    uint8_t a;           // Offset 0 (1 byte)  
    uint16_t b;          // Offset 1 (2 bytes)  
    uint32_t c;          // Offset 3 (4 bytes)  
} PackedStruct_t;
```

Nguyên lý tối ưu hóa

- ▶ Sắp xếp các thành viên của Struct theo thứ tự kích thước **giảm dần** (ví dụ từ uint64_t → uint32_t → uint16_t → uint8_t).
- ▶ Giúp Compiler tự động căn chỉnh tự nhiên mà không cần chèn thêm nhiều padding byte dư thừa.
- ▶ Tiết kiệm bộ nhớ RAM mà không làm giảm tốc độ truy cập của CPU.

Ví dụ so sánh (struct_padding.c)

```
// BAD: Mixed member sizes -> 24 bytes
typedef struct {
    uint32_t a; // 4 bytes
    uint16_t b; // 2 bytes
    uint8_t c; // 1 byte (3 padding bytes)
    uint16_t d; // 2 bytes
    uint64_t e; // 8 bytes
} BadStruct_t;

// GOOD: Decreasing member sizes -> 16 bytes
typedef struct {
    uint64_t e; // 8 bytes
    uint32_t a; // 4 bytes
    uint16_t b; // 2 bytes
    uint16_t d; // 2 bytes
    uint8_t c; // 1 byte (1 padding byte at the end)
} GoodStruct_t;
```

Union hoạt động như thế nào?

- ▶ Tất cả các thành viên trong Union chia sẻ chung **một vùng nhớ vật lý duy nhất**.
- ▶ Kích thước của Union bằng kích thước của thành viên lớn nhất.

Ví dụ thực tế (union_bitfields.c)

```
typedef union {  
    float    fval;  
    uint32_t uval;  
    uint8_t  bytes[4];  
} FloatConverter_t;  
  
FloatConverter_t conv;  
conv.fval = 3.14f;  
  
// Extract bytes for UART transmission  
uint8_t b0 = conv.bytes[0];  
uint8_t b1 = conv.bytes[1];  
uint8_t b2 = conv.bytes[2];  
uint8_t b3 = conv.bytes[3];
```



Bộ nhớ không biến đổi: Flash (ROM)

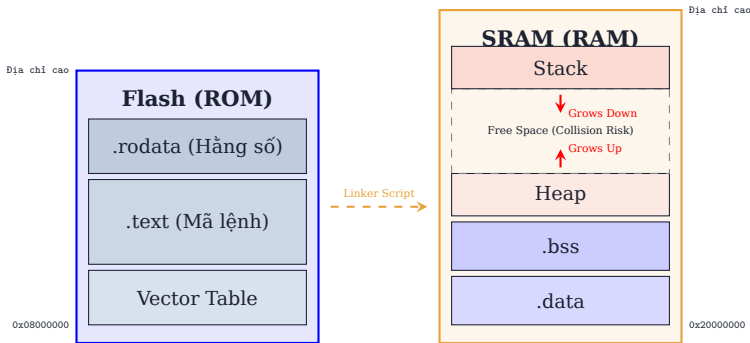
- ▶ **.text:** Chứa mã máy sau khi biên dịch (Instructions).
- ▶ **.rodata:** Chứa hằng số chỉ đọc (const) và chuỗi ký tự literal.

Bộ nhớ biến đổi: SRAM

- ▶ **.data:** Biến toàn cục / static đã khởi tạo giá trị khác 0.
- ▶ **.bss:** Biến toàn cục / static chưa khởi tạo hoặc khởi tạo bằng 0 (được xóa về 0 tại startup).

Phân vùng Stack & Heap (Có thể mở rộng)

- ▶ **Stack:** Lưu biến cục bộ, tham số hàm, địa chỉ phản hồi. Tự động mở rộng **grows down** (phát triển từ địa chỉ cao xuống địa chỉ thấp).
- ▶ **Heap:** Dùng cho cấp phát động. Mở rộng tự do **grows up** (phát triển từ địa chỉ thấp lên địa chỉ cao).
- ▶ **Nguy cơ va chạm (Collision):** Vì Stack và Heap phát triển ngược chiều nhau, nếu không giới hạn, chúng sẽ ghi đè vào nhau gây hư hại dữ liệu nghiêm trọng.



Tại sao nên tránh malloc trong MCU?

- ▶ **Phân mảnh bộ nhớ (Fragmentation):** Cấp phát/giải phóng liên tục tạo ra nhiều "lỗ thủng" nhỏ trong RAM. Khi cần cấp phát một khối lớn liền kề, malloc sẽ thất bại dù tổng RAM trống vẫn đủ.
- ▶ **Thời gian chạy không xác định (Non-deterministic):** Thuật toán tìm kiếm khối trống của malloc tốn thời gian thay đổi tùy theo trạng thái phân mảnh → Không phù hợp với hệ thống thời gian thực (Real-time).

Rủi ro và Giải pháp thay thế

- ▶ **Rủi ro rò rỉ (Memory Leak):** Vì điều khiển chạy liên tục 24/7. Chỉ cần một lỗi nhỏ quên `free()`, RAM sẽ bị cạn kiệt và gây crash hệ thống sau vài ngày/tuần.
- ▶ **Giải pháp thay thế:**
 - ▷ **Cấp phát tĩnh (Static Allocation):** Khai báo mảng tĩnh toàn cục hoặc static ngay từ đầu để quản lý dung lượng RAM cố định.
 - ▷ **RTOS Memory Management:** Sử dụng bộ quản lý Heap chuyên dụng của RTOS (ví dụ: thuật toán `heap_4` của FreeRTOS giúp gộp các khối trống liền kề để chống phân mảnh).

Ngắt (Interrupt) & NVIC

- ▶ **NVIC** (Nested Vectored Interrupt Controller): Bộ điều khiển ngắt lồng nhau của ARM Cortex-M, quản lý các vector ngắt và độ ưu tiên.
- ▶ **Vector Table**: Bảng chứa địa chỉ bắt đầu của các hàm ISR nằm ở địa chỉ đầu tiên của Flash.

Hàm ISR (Interrupt Service Routine)

- ▶ Hàm phục vụ ngắt đặc biệt được kích hoạt trực tiếp bởi phần cứng CPU khi có sự kiện (cạnh tín hiệu GPIO, UART nhận byte, Timer tràn...).

Quy trình xử lý Ngắt của CPU

1. **Sự kiện ngắt xảy ra**: Thiết bị ngoại vi gửi tín hiệu lên NVIC.
2. **Context Save (Lưu ngữ cảnh)**: CPU tự động lưu các thanh ghi hiện tại (PC, LR, SP, R0-R3...) vào Stack.
3. **Vector Table Lookup**: CPU lấy địa chỉ hàm ISR.
4. **Thực thi ISR**: Chạy mã xử lý ngắt.
5. **Context Restore**: Khôi phục trạng thái cũ từ Stack và tiếp tục chạy tiếp code chính.

Đơn luồng vs Đa luồng

- ▶ **Đơn luồng (Bare-metal superloop):** Chương trình chạy trong một vòng lặp 'while(1)'. Nếu một tác vụ bị nghẽn (blocking), cả hệ thống dừng lại.
- ▶ **Đa luồng (Multithreading):** Hệ thống được chia thành nhiều tác vụ độc lập (**Tasks / Threads**) chạy đồng thời.

RTOS (Real-Time Operating System)

- ▶ Hệ điều hành thời gian thực cung cấp cơ chế đa nhiệm, lập lịch và đồng bộ hóa cho vi điều khiển (như FreeRTOS, Zephyr).

Đa nhiệm trên MCU 1 Nhân (Core)

- ▶ Bản chất vi điều khiển 1 nhân chỉ chạy được **1 dòng lệnh duy nhất tại một thời điểm**.
- ▶ Đa nhiệm có được là nhờ CPU chuyển đổi luân phiên giữa các Task với tốc độ cực kỳ nhanh (Concurrency - Đồng thời giả lập), tạo cảm giác các tác vụ đang chạy song song (Parallelism).

Scheduler (Bộ lập lịch)

- ▶ Thành phần cốt lõi của RTOS, quyết định Task nào được quyền sử dụng CPU tại một thời điểm dựa trên thuật toán lập lịch.
- ▶ **Lập lịch cướp quyền (Preemptive):** Task có độ ưu tiên cao hơn sẽ ngay lập tức "cướp" CPU từ Task thấp hơn.
- ▶ **Lập lịch hợp tác (Cooperative):** Task tự nguyện nhường CPU (qua hàm yield/delay).

Chuyển đổi ngữ cảnh (Context Switch)

- ▶ Quá trình dừng một Task đang chạy và tải một Task khác lên để thực thi.
- ▶ **Các bước thực hiện:**
 1. Kích hoạt ngắt hệ thống (như SysTick hoặc PendSV).
 2. **Lưu trạng thái Task cũ:** Đẩy các thanh ghi CPU hiện tại vào vùng Stack riêng của Task cũ.
 3. Lưu con trỏ Stack Pointer (SP) của Task cũ vào Task Control Block (TCB).
 4. **Tải trạng thái Task mới:** Đọc SP của Task mới từ TCB của nó.
 5. Phục hồi các thanh ghi CPU từ vùng Stack của Task mới.

Con trỏ hàm & Task

- ▶ Trong RTOS, một Task thực chất là một **hàm C không bao giờ kết thúc** (chứa vòng lặp vô hạn).
- ▶ Scheduler quản lý các Task này bằng cách lưu địa chỉ của chúng dưới dạng **con trỏ hàm** trong cấu trúc quản lý (TCB).
- ▶ Khi khởi chạy hoặc chuyển đổi task, bộ lập lịch sẽ gọi thực thi gián tiếp qua con trỏ hàm đó.

Ví dụ: Cách Scheduler gọi Task

```
// Define function pointer type for Task
typedef void (*TaskFunction_t)(void *pvParameters);

// Task definition
void vLedTask(void *params) {
    while (1) {
        toggle_led();
        vTaskDelay(100);
    }
}

// Scheduler executes task indirectly
void run_task(TaskFunction_t pFunc, void *param) {
    if (pFunc != NULL) {
        pFunc(param); // Call via pointer
    }
}

int main() {
    // Register and run task
    run_task(vLedTask, NULL);
}
```

Khai báo Task chuẩn (FreeRTOS)

- ▶ Nguyên mẫu: `void TaskFunc(void *params);`
- ▶ Con trỏ hàm: `void (*TaskFunction_t)(void *)`

Tại sao dùng con trỏ void *params?

- ▶ Nhận địa chỉ của **bất kỳ kiểu dữ liệu nào** (cấu trúc config, biến đơn, mảng) mà không cần thay đổi chữ ký hàm của Task.
- ▶ **Ép kiểu con trỏ (Pointer Casting)**: Bên trong Task, ta ép kiểu con trỏ 'void *' ngược lại cấu trúc mong muốn để lấy dữ liệu cấu hình.
- ▶ **Khả năng tái sử dụng**: Giúp chạy cùng

Ví dụ: Truyền Struct tham số vào Task

```
// Configuration structure
typedef struct {
    uint16_t pin;
    uint32_t delay_ms;
} LedConfig_t;

// LED task accepting custom configuration
void vLedTaskCustom(void *params) {
    // Cast void pointer back to config struct
    LedConfig_t *config = (LedConfig_t *)params;
    while (1) {
        toggle_pin(config->pin);
        vTaskDelay(config->delay_ms);
    }
}

int main() {
    LedConfig_t led1 = {.pin = 12, .delay_ms = 500};
    LedConfig_t led2 = {.pin = 13, .delay_ms = 1000};

    // Start tasks with custom configuration arguments
    xTaskCreate(vLedTaskCustom, "LED1", 128, &led1, 1, NULL);
    xTaskCreate(vLedTaskCustom, "LED2", 128, &led2, 1, NULL);
}
```

Vấn đề của Switch-Case lớn

- ▶ Tạo mã nguồn cồng kềnh, khó bảo trì khi thêm chức năng mới.
- ▶ Tốc độ thực thi giảm khi có quá nhiều nhánh rẽ.

Giải pháp Table-driven

- ▶ Ánh xạ sự kiện (Event Index) trực tiếp sang một mảng các con trỏ hàm.
- ▶ Tránh hoàn toàn việc so sánh nhánh rẽ, độ phức tạp đạt $\mathcal{O}(1)$.

Ví dụ menu LCD (table_driven.c)

```
void menu_home(void) { lcd_show("Home"); }
void menu_settings(void) { lcd_show("Settings"); }
void menu_about(void) { lcd_show("About"); }

// Array of function pointers (jump table)
typedef void (*menu_func_t)(void);
static const menu_func_t menu_table[] = {
    menu_home,
    menu_settings,
    menu_about
};

// O(1) menu navigation function
void navigate_menu(uint8_t menu_index) {
    if (menu_index < 3) {
        menu_table[menu_index](); // Indirect call
    }
}
```

Callback trong Ngắt & RTOS

- ▶ Cho phép Driver phần cứng (HAL) giao tiếp ngược với tầng ứng dụng một cách độc lập thông qua đăng ký con trỏ hàm.
- ▶ **Ứng dụng trong RTOS:** Dùng cho các hàm Callback của phần mềm định thời (Software Timers), hoặc hàm xử lý sự kiện ngắt từ hàng đợi (Queue callbacks).

Ví dụ: Driver nhiệt độ (callback_demo.c)

```
// ===== DRIVER LAYER =====
typedef void (*temp_cb_t)(uint16_t raw_val);
static temp_cb_t on_overtemp = NULL;

// Register application callback
void Temp_RegisterCallback(temp_cb_t cb) {
    on_overtemp = cb;
}

// Call registered callback function upon overtemp interrupt
void Temp_Interrupt_Handler(uint16_t val) {
    if (val > 1000 && on_overtemp != NULL) {
        on_overtemp(val); // Notify application layer
    }
}

// ===== APPLICATION LAYER =====
void emergency_shutdown(uint16_t temp) {
    // Perform emergency shutdown procedure
}

int main() {
    Temp_RegisterCallback(emergency_shutdown);
}
```

Semaphore là gì?

- ▶ Công cụ đồng bộ hóa dựa trên giá trị đếm (Token).
- ▶ Task muốn chạy phải lấy token (take), chạy xong trả lại token (give).

Binary Semaphore (0 hoặc 1)

- ▶ Thường dùng để **đồng bộ hóa giữa ISR và Task**.
- ▶ Ví dụ: ISR sau khi nhận đủ gói tin UART sẽ 'give' Semaphore để kích hoạt Task xử lý ở main loop.

Counting Semaphore (> 1)

- ▶ Dùng để quản lý một tài nguyên có số lượng giới hạn.
- ▶ Ví dụ: Đếm số vị trí trống trong Buffer nhận dữ liệu.

Sử dụng Binary Semaphore trong FreeRTOS

```
SemaphoreHandle_t xSem = NULL;

void UART_ISR(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    // Release semaphore from ISR to unblock task
    xSemaphoreGiveFromISR(xSem, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void vProcessingTask(void *pvParameters) {
    xSem = xSemaphoreCreateBinary();
    while (1) {
        // Wait indefinitely until unblocked by ISR
        if (xSemaphoreTake(xSem, portMAX_DELAY) == pdTRUE) {
            process_uart_data(); // Process data
        }
    }
}
```

Mutex (Mutual Exclusion)

- ▶ Cơ chế loại trừ tương hỗ để bảo vệ tài nguyên dùng chung khỏi Race Condition (Tranh chấp).
- ▶ **Quyền sở hữu (Ownership):** Chỉ Task nào giữ Mutex mới được phép giải phóng nó.

Nghịch đảo độ ưu tiên (Priority Inversion)

- ▶ Task ưu tiên Cao (H) phải đợi Task Thấp (L) giải phóng khóa Mutex. Nhưng L lại bị Task Trung bình (M) chiếm quyền CPU → H gián tiếp bị chặn bởi M.
- ▶ **Priority Inheritance (Kế thừa):** Tự động nâng độ ưu tiên của L lên bằng H khi H đang đợi khóa của L → L giải phóng nhanh.

Độc quyền cổng UART bằng Mutex (FreeRTOS)

```
SemaphoreHandle_t xMutex = NULL;

void vPrintTask(void *pvParameters) {
    xMutex = xSemaphoreCreateMutex();
    while (1) {
        // Acquire Mutex to gain exclusive access to UART
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
            printf("Exclusive access to UART acquired...\n");
            // Perform safe print operations
            xSemaphoreGive(xMutex); // Release Mutex
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Queue là gì?

- ▶ Hàng đợi truyền tin an toàn theo cơ chế **FIFO (First In First Out)** giữa các Task hoặc giữa ISR và Task.
- ▶ Tự động đồng bộ: Task ghi bị chặn khi Queue đầy; Task đọc bị chặn khi Queue trống.

Cơ chế Copy (Pass-by-Value)

- ▶ RTOS sao chép hoàn toàn giá trị của dữ liệu vào bộ đệm riêng của Queue.
- ▶ Đảm bảo an toàn bộ nhớ vì Task gửi có thể sửa/ghi đè biến nguồn ngay sau khi gửi.

Gửi và nhận dữ liệu ADC giữa các Task (FreeRTOS)

```
QueueHandle_t xQueue = NULL;

void vSenderTask(void *pvParameters) {
    uint32_t sensor_data;
    while (1) {
        sensor_data = read_adc();
        // Send data copy to queue, block up to 10 ticks if full
        xQueueSend(xQueue, &sensor_data, 10);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void vReceiverTask(void *pvParameters) {
    uint32_t received_val;
    while (1) {
        // Wait indefinitely until data is received
        if (xQueueReceive(xQueue, &received_val, portMAX_DELAY) == pdTRUE) {
            process_data(received_val); // Process received value
        }
    }
}
```

Vùng Stack riêng biệt của từng Task

- ▶ Khác với hệ thống Bare-metal chỉ có 1 vùng Stack dùng chung cho toàn bộ chương trình.
- ▶ Trong hệ thống RTOS, **mỗi Task bắt buộc phải có một vùng Stack riêng biệt**.
- ▶ Vùng Stack này được cấp phát khi tạo Task (thường nằm trên Heap của RTOS).

Stack chứa những gì?

- ▶ Biến cục bộ trong các hàm của Task.
- ▶ Tham số truyền vào hàm.
- ▶ Ngữ cảnh CPU (Context) khi Task bị dừng.

Thiết lập kích thước Stack (Stack Size)

- ▶ Thiết lập quá nhỏ → Dễ dẫn đến **Stack Overflow** (Tràn ngăn xếp) khi Task gọi hàm sâu hoặc chứa mảng local lớn → Hệ thống crash ngay lập tức.
- ▶ Thiết lập quá lớn → Lãng phí tài nguyên RAM cực kỳ hạn chế của MCU.
- ▶ **Cách tối ưu:** Sử dụng tính năng giám sát kích thước Stack của RTOS (ví dụ: `uxTaskGetStackHighWaterMark()` trong FreeRTOS) để đo lường Stack trống nhỏ nhất thực tế, từ đó căn chỉnh phù hợp.

Quá trình liên kết (Linking)

- ▶ Là bước cuối cùng sau khi Compiler/Assembler biên dịch các file nguồn '.c' thành các file mã vật lý riêng lẻ '.o' (Object files).
- ▶ Các file '.o' chứa mã máy nhưng các hàm/biến được trỏ bằng địa chỉ tương đối (chưa có vị trí vật lý thật).
- ▶ Linker có nhiệm vụ gom toàn bộ các file '.o', file thư viện hệ thống lại và giải quyết các tham chiếu chéo (Cross-references) giữa các file.

Linker Script (.ld)

- ▶ Bản thiết kế hướng dẫn Linker cách sắp đặt các phân vùng '.text', '.data', '.bss', 'stack', 'heap' vào đúng các dải địa chỉ vật lý của bộ nhớ **Flash** (ROM) và **SRAM** trên vi điều khiển.
- ▶ Định nghĩa chính xác dung lượng bộ nhớ thực tế của chip (ví dụ: Flash bắt đầu từ '0x08000000', RAM từ '0x20000000').
- ▶ Sản phẩm cuối cùng của Linker là file executable ('.elf', '.hex', '.bin') để nạp trực tiếp xuống chip.

- ▶ **MCU vs MPU & Hệ thống Ngắt:** Làm chủ cấu trúc phần cứng và cơ chế ngắt NVIC giúp phản hồi tức thời sự kiện với mã nguồn ISR tối ưu, reentrant.
- ▶ **Struct, Union & Bit-fields:** Giải quyết vấn đề căn chỉnh dữ liệu, tối ưu hóa bộ nhớ RAM và thiết lập driver thành ghi trực tiếp bằng bit-fields.
- ▶ **Phân vùng bộ nhớ & Cấp phát động:** Phân bổ SRAM hợp lý (.data, .bss, stack, heap) và lý do hạn chế dùng malloc để tránh lỗi phân mảnh RAM trên MCU.
- ▶ **Đa nhiệm & Bộ lập lịch trên MCU:** Hiểu cách hoạt động phân chia thời gian chạy CPU và cơ chế Context Switch của Scheduler để quản lý các tác vụ đa luồng.
- ▶ **Con trỏ Hàm & Callback:** Ứng dụng con trỏ hàm trực tiếp trong việc định nghĩa và gọi Task của Scheduler, thiết kế driver hướng sự kiện với kiến thức callback.
- ▶ **Cơ chế RTOS (Mutex, Semaphore, Queue, Stack):** Đồng bộ hóa và truyền thông tin an toàn giữa các luồng bằng các công cụ Mutex (chống Priority Inversion), Semaphore tín hiệu, Queue truyền tin an toàn, và quản lý Stack chặt chẽ cho từng Task.
- ▶ **Vai trò của Linker:** Hiểu cách Linker Script (.ld) sắp xếp mã máy và biến toàn cục vào đúng địa chỉ vật lý Flash và RAM của vi điều khiển.

Thank You

Acknowledgements

Cám ơn các bạn đã lắng nghe. Hãy thực hành làm chủ lập trình đa nhiệm và thiết kế kiến trúc hệ điều hành thời gian thực tối ưu trên vi điều khiển!